METROPOLITAN
Community College

Databases in Python

Modify and Delete

INFO 1501 – Module 9

Modify in Python

1501

Update a row

- To update a row, the entire table or a selection of rows you use the `UPDATE` command
- You must define the table
- Use the `SET` command to define what columns and what should be updated with an '='
- Include the `WHERE` clause to limit what rows to update

```
UPDATE product
SET stock_qty = 20
WHERE id = 1;

UPDATE product
SET price = price * 1.05;
```

Simple Updates of One Item

- Depending on the system you are building you may need to update an entire row or just a couple of columns
- For example, in our application here one of things we need to constantly update would be the on-hand stock of a product
- This functions returns `True` if successful, `False` if not
- Notice we are accessing `rowcount` which is another property on the cursor of the last `execute()` command how many rows did we update.
  - Potentially if you want you could just return True, but this way it makes sure we updated a row via the id
  - If the id of an item doesn't exist then we don't update and `rowcount` would be 0 making it return `False`

```
def update_product_stock(product_id: int, new_stock_qty: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
            "UPDATE product SET stock_qty = ? WHERE id = ?",
            (new_stock_qty, product_id))
        conn.commit()

        return cursor.rowcount == 1

    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Another Way to Update the Stock

- Maybe we want to not just update the stock the stock but ADD to the stock with what inventory we received.
- Here we can tell SQL to take the value and perform math operations
  - Just like we do in typical Python programming

```
def add_product_stock(product_id: int, add_qty: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
            "UPDATE product SET stock_qty = stock_qty + ? WHERE id = ?",
            (add_qty, product_id))
        conn.commit()

        return cursor.rowcount == 1
    except:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Modify multiple rows

- What happens when we want to update multiple rows
- For that we just adjust our `WHERE` clause or omit it at all
- Here we update all prices of products based on the percentage increase
- We also do some validation like making sure the percentage is > 0
  - Don't want malicious modification/negative values
- Also since SQLite doesn't control decimal places like other databases we use `ROUND()` going to two decimal places

```
def raise_product_price_percentage(percentage : float) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    if percentage <= 0:
        return False # not update/error on percentage

    try:
        cursor.execute(
         "UPDATE product SET price = ROUND(price * ?, 2)", (1 + percentage,))

        conn.commit()

        return cursor.rowcount >= 1

    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Functions to UPDATE each column?

- What if our system allowed for us to update all the columns (or at least most) of a row?
- We could create a function for each
- But we have a lot of repeated code
  - We could send it the column name to update along with the table name?
- But there could be a better option

```
def update_product_name(product_id : int, name : str) -> bool:

def update_product_brand(product_id : int, brand : str) -> bool:

def update_product_price(product_id : int, price : float) -> bool:

def update_review_rating(review_id : int, rating : int) -> bool:

def update_review_title(review_id : int, title : str) -> bool:

def update_review_description(review_id : int, description : str) -> bool:
```

UPDATE Entire Row

- The better approach could be to not worry about what table and column to update and just update the entire item
- For this we have a function that takes our object in the parameter and updates all the columns (that can be updated)
  - We don't want to change the id for example
  - Or a product_id in the review of a product
- The process should go as follows
  - On the UI side you query the database and get that object created and sent back
  - Go through operations of updating the local instance that was created
  - Then call upon this one function, send the modified object to be saved to the table
- What if we only change one column? We are still going to overwrite all the data – columns that don't change will still keep their same data since the object holds that data from the query

```
def update_product(product : Product) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
         """UPDATE product SET name = ?, brand = ?, price = ?, stock_qty = ?
         WHERE id = ? """, (product.name, product.brand, product.price,
             product.stock_qty, product.id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()

def update_review(review : Review) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
         "UPDATE review SET rating = ?, title = ?, comment = ? WHERE id = ?",
            (review.rating, review.title, review.comment, review.id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Coding Example – Modify and Delete

- Modify
  - Updating owner information – just do 1 function here
  - Updating Accounts
    - Balance for debit/credit separately?
    - Everything else in a regular "update everything" function.
  - Updating Account's owner list – would be an INSERT into our accounts_owner table

Delete in Python

INFO 1501

Delete

- The DELETE command can be use to remove rows from a table
- Always be careful with delete to include a WHERE clause otherwise you delete all the data in a table
- Often times in the business environment you will have users for a database and set privileges
  - This way you can't delete data
- You can also use the DROP command on a table to delete the table entirely
- You may not ever want to delete but instead have properties like "active" or "inactive" for accounts
- When deleting if there are any relationships with foreign keys errors can occur
  - In our example, we have review tied to products, if we try to delete a product with reviews then we will get a foreign key constraint error
  - There are some settings we can adjust to CASCADE delete – we won't worry about this now
  - Or just make sure to delete all reviews first before deleting a product

```
DELETE FROM review
WHERE id = 10;
```

```
DROP TABLE review;
```

Simple Deletes

- When deleting we need to make sure that we get some kind of column value to delete by
- Most of the time we get an id of the item in the table
- Again, you should have a WHERE clause in your SQL
- Here on the UI side we got the review by doing a query then sent over the id of that review to be deleted

```
def delete_review(review_id: int)-> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("DELETE FROM review WHERE id = ?", (review_id,))
        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Deleting when foreign keys involved

- Note that the deletion I showed on the last slide was for a review
- This is because if we delete a product that has reviews tied to it, then an error occurred
- For our system here we need to run two deletions
  - one where we delete all reviews that map to the foreign key
  - one delete for the product
- The rowcount property only relates to the last execute() function call, so here we track the first delete and add to the product delete
  - A product could have no reviews so we check greater than equal to 1
- Again depending on your rules and what database you are using, you may not need this logic and it may delete all relationship objects – using `ON DELETE CASCADE`

```
def delete_product(product_id: int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("DELETE FROM review WHERE product_id = ?", (product_id,))
        reviews_rows = cursor.rowcount
        cursor.execute("DELETE FROM product WHERE id = ?", (product_id,))
        conn.commit()

        return (reviews_rows + cursor.rowcount) >= 1

    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Soft Delete to a different table

- Depending on the business rules and industry regulations you may need to actually "keep" the data you have and not actually do a delete
- One way to perform this is moving the data from one table over to a different table as a sort of "archive"
- Typically, we perform this in one function as a "transaction" on the database
  - We don't want insert functions that we query another table and create an object to insert to our archive table – this way we can rollback if there are errors
- In this situation we have our reviews on the products that reference through a foreign key
- Because of this we need to move over not only the product but the reviews as well
  - This can be costly on the database with processing
  - We are only showing this as an example but if we have any data involving foreign keys, this should be avoided
  - This method is good if we don't have any foreign key connections

```
def archive_product(product_id: int)-> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("""INSERT INTO archive_product
                       SELECT id, name, brand, price, stock_qty
                       FROM product WHERE id = ?""", (product_id,))

        cursor.execute("""INSERT INTO archive_review
                   SELECT id, product_id, rating, title, comment, reviewer
                   FROM review WHERE product_id = ?""", (product_id,))

        cursor.execute("DELETE FROM review WHERE product_id = ?",
                                                              (product_id,))

        cursor.execute("DELETE FROM product WHERE id = ?",
                                                              (product_id,))

        conn.commit()

        return True
    except Exception:
        conn.rollback()
        return False
    finally:
        conn.close()
```

Soft Delete – “active” column

- In this situation and maybe better overall would be to add a column that marks if a row of data is “active”
- We can easily do that on our table creation
- But what if we already have our table and there is data in it, we don’t want to `DROP` a table to erase it all
- Instead we can use an `ALTER TABLE` function to `ADD COLUMN`
- Here we add a column and also set the default value to ‘1’ for True
  - This eliminates the need for a `NOT NULL` property as it will automatically put in ‘1’ if we don’t provide it
  - A new product that we add in should be active right?
  - This command puts in ‘1’ as a value for the rows of data we already have

```
CREATE TABLE IF NOT EXISTS product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    brand TEXT NOT NULL,
    price REAL NOT NULL,
    stock_qty INTEGER NOT NULL
    active INTEGER NOT NULL
);
```

```
ALTER TABLE product
ADD COLUMN is_active
INTEGER DEFAULT 1;
```

Soft Delete – python functions

- We have a couple options to go here for deactivate/activate
  - We can create separate functions
- Or if we know we may re-activate an item, then you need to send over the value that you want to set to active

```
def deactivate_product(product_id : int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute(
           "UPDATE product SET is_active = 0 WHERE id = ?", (product_id,))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()

def update_is_active(product_id : int, active : int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()

    try:
        cursor.execute("UPDATE product SET is_active = ? WHERE id = ?",
                                                    (active, product_id))

        conn.commit()

        return cursor.rowcount == 1
    except Exception:
        conn.rollback()
        return False

    finally:
        conn.close()
```

Soft Delete – Update your SELECT

- With this added property for our rows you will need to go back through your SELECT functions where you are always checking for active items
- You will need some functionality to search through inactive rows in your system
- This means adding either a WHERE clause in the your get everything queries or adding in AND to check multiple conditions

```
cursor.execute("SELECT * FROM product WHERE is_active = 1")

cursor.execute("SELECT * FROM product WHERE id = ? AND is_active = ?",
                                                    (product_id,))
```

Coding Example – Modify and Delete

- Delete
  - "Close" accounts – add the active column since we have foreign keys
  - Delete Transactions – this needs to happen like in bank accounts if we have temporary transactions that actually don't "post" or get saved.
    - Could add a column of posted or not – but that is for more advanced features
  - Update our UI to have these features

Object Relation Mapper

INFO 1501

What is an ORM?

- When applications get rather large in their operation writing raw SQL can become repetitive
- As you may experience in the homework and in these lectures, this is a lot of code
- We typically call this "boilerplate code"
  - Code that is repetitive is minimal ability to change or make generic
  - Most database access code is boilerplate
    - It also repeats across different applications
- Instead of writing some SQL to select a row of data, ORMs have functions like .get() that will get from the database for us

Why ORMs?

- They solve several common problems
  - Reduce repetitive SQL code
  - Centralize database logic
  - Improve readability and maintainability
  - Make it easier to change databases later
  - Integrate validation and relationships directly into objects
- They are especially common in web applications
- Most widely used ORM in Python is SQLAlchemy
  - 3rd party not built into Python by default
  - Need to install using pip
- We are not using ORMs here because you need to learn how to directly access via SQL commands
- In future classes you may be using ORMs